Machine Learning Technologies
Support Vector Machine

Donny Hurley

ATU

Recall

$$y = mx + c$$

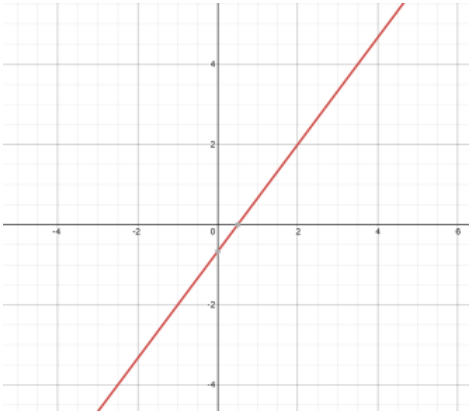
is the equation of a line.

Basically if that equation is satisfied for a point (x, y) then the point falls on the line. We can always write it as $ax + by + c = 0$ for some numbers a, b, c and that will describe a line

$$\mathbf{w}^T\mathbf{x} + b$$

is also an equation of a line, a vector equation of a line. Whatever way you want to think about it, it described a line - is linear.

The following is the plot of the line $4x - 3y - 2 = 0$



- Clearly from the picture $(2, 2)$ falls on the line and $4(2) - 3(2) - 2 = 0$ so the equation is satisfied showing this.
- Look at the image, the point $(2, -2)$ is clearly to the right of the line. Try the equation and get $4(2) - 3(-2) - 2 = 12$. A positive number.
- Now, the point $(-4, 2)$ is clearly to the left of the line. Equation: $4(-4) - 3(2) - 2 = -24$. A negative number.

So whether the calculation of the line is positive or negative will tell us which side of the line it falls on.

Let's extend this

Take

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} \quad \text{and} \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Put it into $\mathbf{w}^T\mathbf{x} + b$

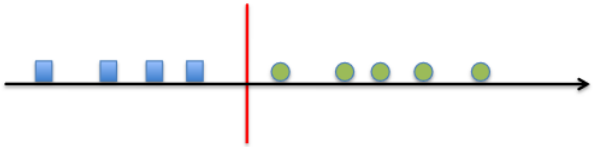
$$\mathbf{w}^T\mathbf{x} + b = \begin{pmatrix} w_1 & w_2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + b = w_1x_1 + w_2x_2 + b$$

If $\mathbf{w} = \begin{pmatrix} 4 \\ -3 \end{pmatrix}$ and $b = -2$ then we have $4x - 3y - 2$

Then if we evaluate the feature $(2, -2)$ we find it is a positive so we classify it in the category of that side of the line, the positive class. $(-4, 2)$ classify in the negative class This expands quite easily to 3,4,5,6,7,8 dimensions so works just fine for how many features we have, we just can't visualise past 3! The result

Definition

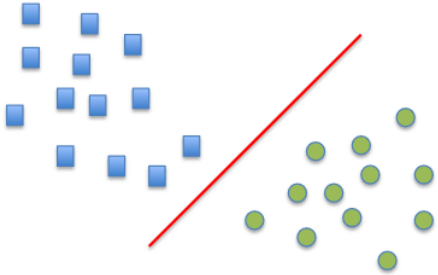
$\mathbf{w}^T\mathbf{x} + b$ describes a hyperplane in any dimension



- Linear classifier has a linear boundary (hyperplane)

$$w_0 + \mathbf{w}^T\mathbf{x} = 0$$

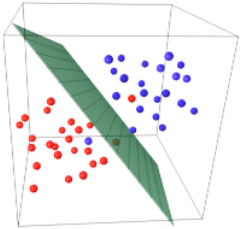
  which separates the space into two "half-spaces"
- In 1D this is simply a threshold



- Linear classifier has a linear boundary (hyperplane)

$$w_0 + \mathbf{w}^T\mathbf{x} = 0$$

  which separates the space into two "half-spaces"
- In 2D this is a line



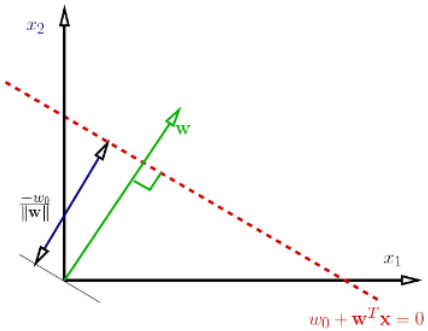
- Linear classifier has a linear boundary (hyperplane)

$$w_0 + \mathbf{w}^\mathsf{T}\mathbf{x} = 0$$

  which separates the space into two "half-spaces"
- In 3D this is a plane

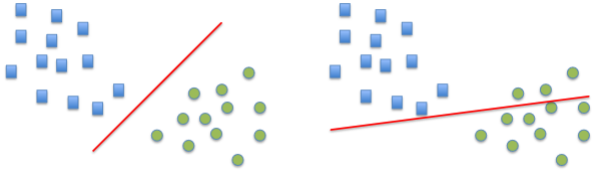
Geometry

$\mathbf{w}^T\mathbf{x} = 0$ is a line passing through the origin and is orthogonal to $\mathbf{w}$
$\mathbf{w}^T\mathbf{x} + w_0 = 0$ shifts it by w_0



Learning Linear Classifiers

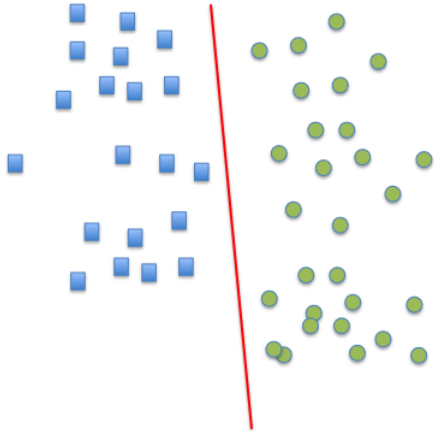
- Learning consists in estimating a "good" decision boundary
- We need to find $\mathbf{w}$ (direction) and w_0 (location) of the boundary
- What does "good" mean?
- Is this boundary good?



- We need a criteria that tell us how to select the parameters - use a loss function.

Separating Classes

If we can separate the classes, the problem is linearly separable



Separating Classes

Causes of non perfect separation:

- Model is too simple
- Noise in the inputs (i.e., data attributes)
- Simple features that do not account for all variations
- Errors in data targets (mis-labellings)

Should we make the model complex enough to have perfect separation in the training data?

Decision Boundaries

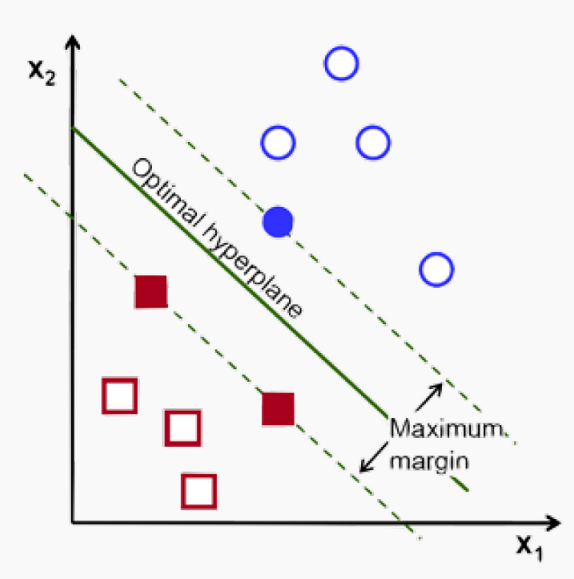
- We can consider alternative constraints that prevent overfitting.
- For example, we may prefer a decision boundary that does not 'favour' any class (esp. when the classes are roughly equally populous).
- Geometrically, this means choosing a boundary that maximizes the distance or margin between the boundary and both classes.

Recall

- For classification we will use a "linear" model

$$y(\mathbf{x}) = f(\mathbf{w}^T\mathbf{x} + w_0)$$

- This is called a *generalised linear model* and f is a fixed non-linear function
- *Decision boundary* between classes. (if it falls above the boundary then class 1, otherwise class -1)
  - We may prefer a decision boundary that does not 'favour' any class (esp. when the classes are roughly equally populous).
  - Geometrically, this means choosing a boundary that maximizes the distance or margin between the boundary and both classes.

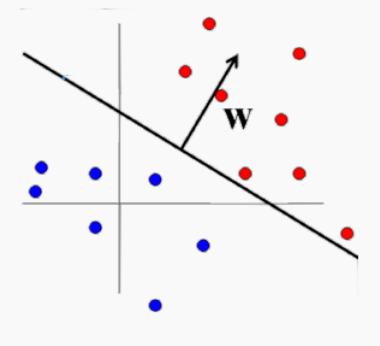

x_2
Optimal hyperplane
Maximum
margin
x_1

Geometry to Decision Boundary

Recall that the decision boundary is defined by some equation in terms of the predictors. A linear boundary is defined by

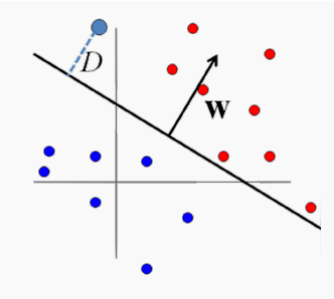
$$\mathbf{w}^T\mathbf{x} + b = 0$$

Recall that the non-constant coefficients, $\mathbf{w}$, represent a **normal vector**, pointing orthogonally away from the plane



Geometry

Now, using some geometry, we can compute the distance between any point to the decision boundary using $\mathbf{w}$ and b.



The signed distance from a point $\mathbf{x} \in \mathbb{R}^n$ to the decision boundary is

$$D(\mathbf{x}) = \frac{\mathbf{w}^T\mathbf{x} + b}{||\mathbf{w}||}$$

Maximising Margins

So our goal. Find a decision boundary that maximises the distance to both classes, i.e. optimise

$$\begin{cases} \max_{w,b} M \\ \text{such that } |D(\mathbf{x^{(i)}})| = \frac{y^{(i)}(\mathbf{w}^T\mathbf{x^{(i)}}+b)}{||\mathbf{w}||} \geq M, i = 1, \ldots, N \end{cases}$$

where M is a real number representing the width of the 'margin' and $y_i = \pm 1$. The inequalities $|D(x_n)| \geq M$ are called constraints.

Maximising Margins

So it is not enough to just choose **w** that creates a decision boundary, we now want an 'optimal' boundary.

The previous formula is more complicated than it needs to be. Maximising the distance of all points to the boundary, is the same as maximising the distance to the **closest points**.

The points closest to the decision boundary are called **support vectors**.

For any plane, we can always scale the equation

$$w^T x + b = 0$$

so that the support vectors lie on the planes

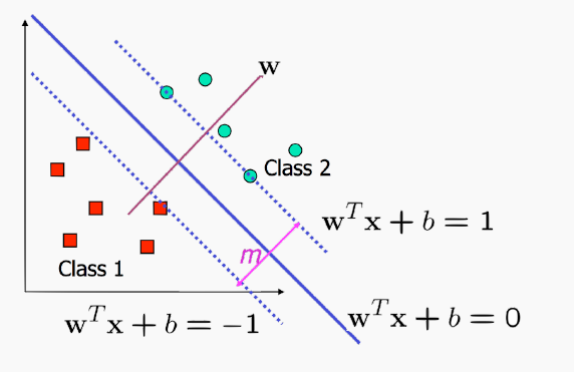
$$w^T x + b = \pm 1$$

depending on their classes.

Maximising Margins

For points on planes $w^T x + b = \pm 1$, their distance to the decision boundary is $\pm \frac{1}{||w||}$.

So we can define the **margin** of a decision boundary as the distance to its support vectors, $m = \frac{2}{||w||}$



Support Vector Classifier: Hard Margin

Finally, we can reformulate our optimization problem - find a decision boundary that maximises the distance to both classes - as the maximisation of the margin, M, while maintaining zero misclassifications

$$\begin{cases} \max_{w,b} \frac{2}{||w||} \\ \text{such that } y^{(i)}(w^T x^{(i)} + b) \geq 1, i = 1, \ldots, N \end{cases}$$

You can also look at it as minimising $||w||^2$

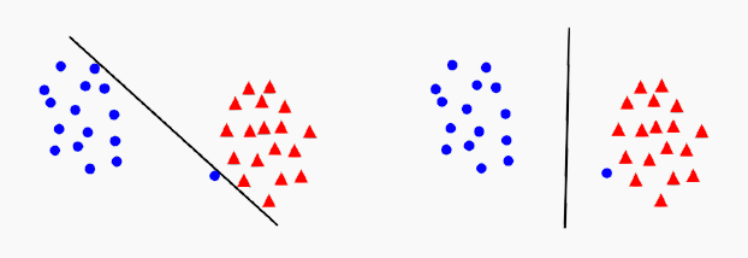
$$\min_{w} ||\mathbf{w}||^2 \text{ such that } y^{(i)}(w^T x^{(i)} + b) \geq 1, i = 1, \ldots, N$$

This is a quadratic optimisation problem, has linear constraints and there is a unique solution.

Calculus again! (Lagrange multipliers if you want to look up the maths)

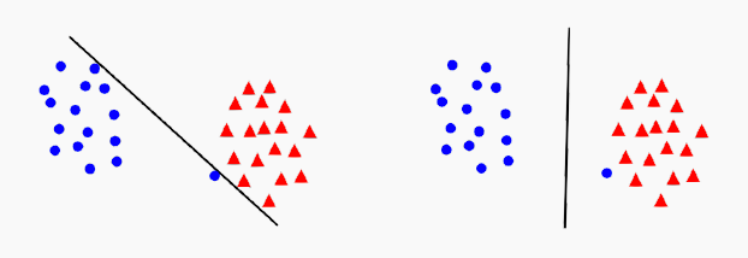
Margin Error/Trade off

Which one of these lines is a better generalisation?



- In the first one the points can be linearly separated but there is a very narrow margin
- But possibly the large margin solution is better, even though one constraint is violated

Margin Error/Trade off

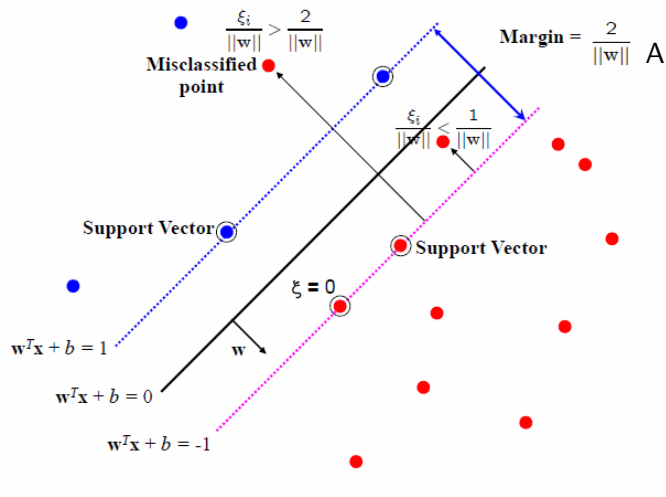


Maximising the margin is a good idea as long as we assume that the underlying classes are linear separable and that the data is noise free.

If data is noisy, we might be sacrificing generalisability in order to minimise classification error with a very narrow margin

With every decision boundary, there is a trade-off between maximising margin and minimising the error.

"Slack Variables"



A variable $\xi_i \geq 0$ for each point.

- For $0 < \xi \leq 1$ point is between margin and correct side of hyperplane. This is called a margin violation
- For $\xi \geq 1$ point is misclassified
- For $\xi = 0$ point is the correct side of the margin.

"Soft Margin" Solution

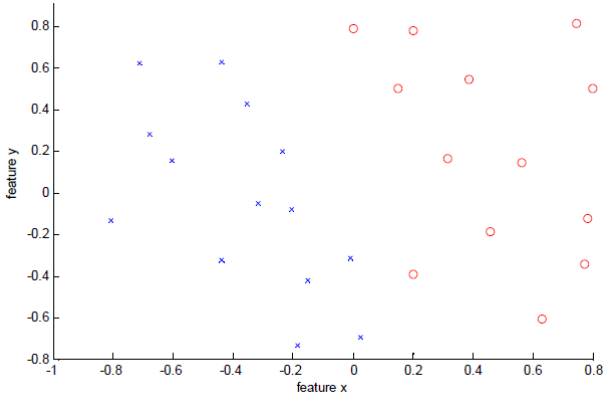
Our problem is re framed as

$$\min_{w,b} ||w||^2 + C \sum_{i}^{N} \xi_i$$

$$\text{such that } y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, i = 1, \ldots, N$$

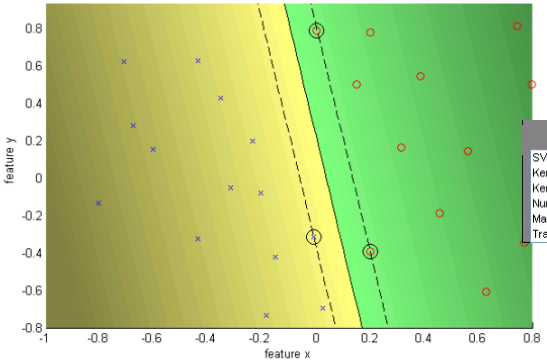
- C is a regularisation parameter: (some notes will use λ instead of C, scikit-learn uses C)
  - small C allows constraints to be easily ignored $\rightarrow$ large margin
  - large C makes constraints hard to ignore $\rightarrow$ narrow margin
  - C $\rightarrow \infty$ enforces all constraints: hard margin
- This is still a quadratic optimization problem and there is a unique minimum. Note, there is only one parameter, C (that you choose).

Pictures



- Data is linearly separable
- But only with a narrow margin

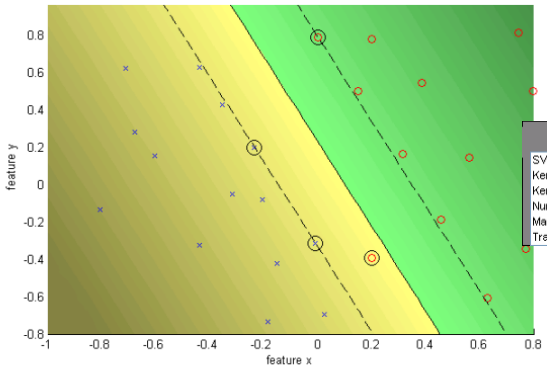
Infinity C - Hard Margin



Comment Window

SVM (L1) by Sequential Minimal Optimizer
Kernel: linear (-), C: Inf
Kernel evaluations: 971
Number of Support Vectors: 3
Margin: 0.0966
Training error: 0.00%

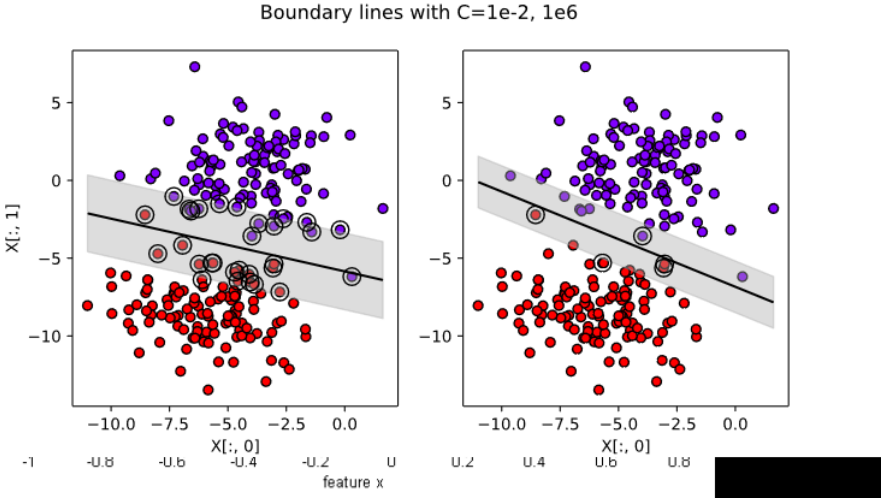
$C = 10$ - Soft Margin



Comment Window

SVM (L1) by Sequential Minimal Optimizer
Kernel: linear (-), C: 10.0000
Kernel evaluations: 2645
Number of Support Vectors: 4
Margin: 0.2265
Training error: 3.70%

Example



Example

In this case, smaller C does slightly better.

```
model = SVC(kernel='linear', C=1e-2)
model.fit(X_train, y_train)
print(model.score(X_valid, y_valid))

model = SVC(kernel='linear', C=1e6)
model.fit(X_train, y_train)
print(model.score(X_valid, y_valid))

0.986666666667
0.973333333333
```

In general, the best C parameter depends on the situation. Experiment. (Cross-Validation?) One note: larger C takes more computation to train.

Previous problem - Breast Cancer Data Set

SVM classifiers often do better on the "hard" problems. If we return to the breast cancer data set:

```
model = make_pipeline(
    StandardScaler(),
    SVC(kernel='linear', C=2.0)
)
model.fit(X_train, y_train)
print(model.score(X_valid, y_valid))

0.993006993006993
```

That's better than either GaussianNB or KNeighborsClassifier.